实验十 JMeter 压测

一、 目的与要求

学习 JMeter 压力测试方法，使用 JMeter 测试 Sentinel

二、 实验内容

2.1 学习 JMeter 压力测试方法
2.2 使用 JMeter 测试 Sentinel

三、 实验指导

3.1 搭建 JMeter 环境

jmeter 官网地址：https://jmeter.apache.org

1.下载压缩包步骤

```
→打开链接：https://jmeter.apache.org/download_jmeter.cgi
→找到：Apache JMeter 5.6.2 (Requires Java 8 or 9.)
→找到：Binaries
→选中：apache-jmeter-5.6.2.zip
→点击下载到任意磁盘，解压到英文目录下。
```

2.配置环境变量

新建 JMETER_HOME：配置为 JMeter 安装的根目录

修改 Path：新增 JMeter 的 bin 目录

修改 ClassPath：新增

;%JMETER_HOME%\lib\ext\ApacheJMeter_core.jar;%JMETER_HOME%\lib\jorphan.jar;

3.检查安装是否成功

进入 jmeter 下的 bin 目录，点击打开 jmeter.bat 文件： 成功启动 jmeter 则表示成功安装 jmeter，如果出现启动闪退，则将系统环境变量中的变量名为 JMETER_HOME 的系统变量删除，再启动该文件即可。

3.2 JMeter 汉化

修改配置文件 jmeter.properties ，添加以下字段：

language = zh_cn

然后重启 JMeter，即可完成汉化。

3.3 JMeter 测试排队等待效果

排队等待方式不会拒绝请求，而是严格控制请求通过的间隔时间，也就是让请求匀速通过。

1. 在 Sentinel 控制台设置流控规则为排队等待

2. 使用 JMeter 测试效果

右键添加一个线程组，修改名称为测试排队等待，设置线程数为 100（也就是 100 个请求），Ramp-Up 时间设置为 10（也就是 100 个请求在 10 秒内发完）

右键“线程组”，添加取样器，选择 HTTP 请求，修改协议为 http、IP、端口号、请求方式、请求路径为/worker-serv/worker

右键“取样器”，添加监听器，选择查看结果树。

3. 启动 JMeter

观察 Sentinel 控制台效果。

时间	通过 QPS	拒绝QPS	响应时间 (ms)
17:36:45	4.0	0.0	4,995.0
17:36:44	5.0	0.0	4,994.0
17:36:43	5.0	0.0	4,997.0
17:36:42	5.0	0.0	4,977.0
17:36:41	5.0	0.0	4,978.0
17:36:40	5.0	5.0	4,801.0

4. 计算结果

QPS 为 5，我们设置 100 个请求 10 秒发完，所以在 10 秒内，一共可以处理 50 个请求。同时，排队队列超时时间为 5 秒，也就是说还有 25 个请求可以在排队等待。

所以整体运行效果为 100 个请求中，正确处理 75 个请求，丢弃 25 个请求。（可能存在略微的误差）

将流控规则修改为等待超时时间为 10 秒，再次启动 JMeter，发现所有请求均可以正确完成。

5. 应用场景

排队等待的规则应用最广泛的就是削峰！比如双十一、选课，为什么有些请求一直会失败，就是被丢弃了。

3.4 JMeter 测试关联流控效果

当两个资源具有依赖关系或者资源争抢，比如一个从数据库中读取数据，一个从数据库中写入数据。读取过高影响写入效率，写入过高影响读取效率，如果说放任不管，本身争抢这个过程也会降低整体的吞吐量。

通过关联流控，我们可以设置限流，从而达到写优先的目的。这样当写操作过于频繁的时候，限制读操作的流量。

测试接口为：

http://localhost:8088/worker-serv/worker

和 http://localhost:8088/worker-serv/worker/workerList

给/worker-serv/worker 设置流控如下：

含义是当关联资源/worker/workerList 的 QPS 达到 1，则限制/worker 的访问

使用 JMeter 测试

右键添加一个线程组，修改名称为测试排队等待，设置线程数为 100（也就是 100 个请求），Ramp-Up 时间设置为 20（也就是 100 个请求在 20 秒内发完）

右键“线程组”，添加取样器，选择 HTTP 请求，修改协议为 http、IP、端口号、请求方式、请求路径为/worker-serv/worker/workerList

右键“取样器”，添加监听器，选择查看结果树。

执行 JMeter，马上回到浏览器请求 http://localhost:8088/worker-serv/worker
发现被限流

3.5 JMeter 测试链路流控效果

1. 在 Worker 服务中，创建一个 SentinelService，代码如下：
com.etoak.java.service.impl 包下

```
/***
 * @Author 高俊 QQ:1120934832
 * @Slogan 易途科技，精英启航
 */
@Service
public class NodeService {

    @SentinelResource(value = "nodeA",blockHandler = "nodeABlockHandler")
    public String commonResource(String from){
        System.out.println("来自入口 " + from + "的请求");
        return from;
    }

    public String nodeABlockHandler(String from, BlockException e){
        return "资源被限流了";
    }
}
```

2. 在 WorkerController 中创建两个接口都调用此服务

```
@RequestMapping("/entrance1")
public ResultVO entrance1(){
    String result
            = nodeService.commonResource("entrance1");
    return ResultVO.success(result);
}
@RequestMapping("/entrance2")
public ResultVO entrance2(){
    String result
            = nodeService.commonResource("entrance2");
    return ResultVO.success(result);
}
```

3. 启动 sentinel 控制台，请求两个接口，刷新控制台页面，观看效果。
会发现只显示一个 NodeA 服务

4. 修改 Worker 服务的 bootstrap 配置文件
添加以下配置：

```
sentinel:
  transport:
    dashboard: localhost:8080
  web-context-unify: false
```

重新请求接口，并刷新 sentinel 查看效果。

5.给其中任意一个 NodeA 服务添加流控如下：

6.当 entrance1 入口 QPS 超过 1，该入口会被限流，但是 entrance2 的入口不会被限流。

3.6 JMeter 测试熔断 - 慢调用比例

参考 Sentinel 中测试熔断的过程。

测试接口：worker-serv/worker

1.设置熔断策略

打开 Sentinel 面板，在簇点链路中，找到/worker，然后添加熔断。如下图：

*：

最大 RT：可以理解为最大 Response Time 最大返回时间

比例阈值：0.5，慢调用的请求比例超过 0.5 即熔断

熔断时长：20，触发熔断后，服务在 20s 内不可用

统计时长+最小请求数：在统计时间内，请求数量>最小请求数才会使熔断生效

2.给接口添加一个延迟时间

```
Thread.sleep(300);
```

3.启动 JMeter

添加测试计划,右键添加一个线程组，修改名称为测试排队等待，设置线程数为 100（也就是 100 个请求），Ramp-Up 时间设置为 9（也就是 100 个请求在 9 秒内发完）

右键“线程组”，添加取样器，选择 HTTP 请求，修改协议为 http、IP、端口号、请求方式、请求路径为/worker-serv/worker

右键“取样器”，添加监听器，选择查看结果树。

3.7 JMeter 测试熔断 - 异常比例

测试接口：worker-serv/worker

1.设置熔断策略

打开 Sentinel 面板，在簇点链路中，找到/worker，然后添加熔断。如下图：

2.给接口主动抛出一个异常

各种方式都可以，比如 Integer.parseInt(“高”)； 1/0

3.启动 JMeter

添加测试计划,右键添加一个线程组，修改名称为测试排队等待，设置线程数为 100（也就是 100 个请求），Ramp-Up 时间设置为 9（也就是 100 个请求在 9 秒内发完）

右键“线程组”，添加取样器，选择 HTTP 请求，修改协议为 http、IP、端口号、请求方式、请求路径为/worker-serv/worker

右键“取样器”，添加监听器，选择查看结果树。

3.8 扩展：开启 Feign 对 Sentinel 的支持

1.在 WorkerController 中添加一个查询用户列表的接口

```
/**
 * 查询用户列表数据
 * @return
 */
@RequestMapping("/workerList")
public ResultVO getWorkerList(){
    return ResultVO.success(workerService.list());
}
```

2.修改 Engineer 工程中的 IWorkerFeign.java，如下：

```
@FeignClient(name = "worker-service",path = "/worker",
        configuration = FeignLoggerConfig.class)
```

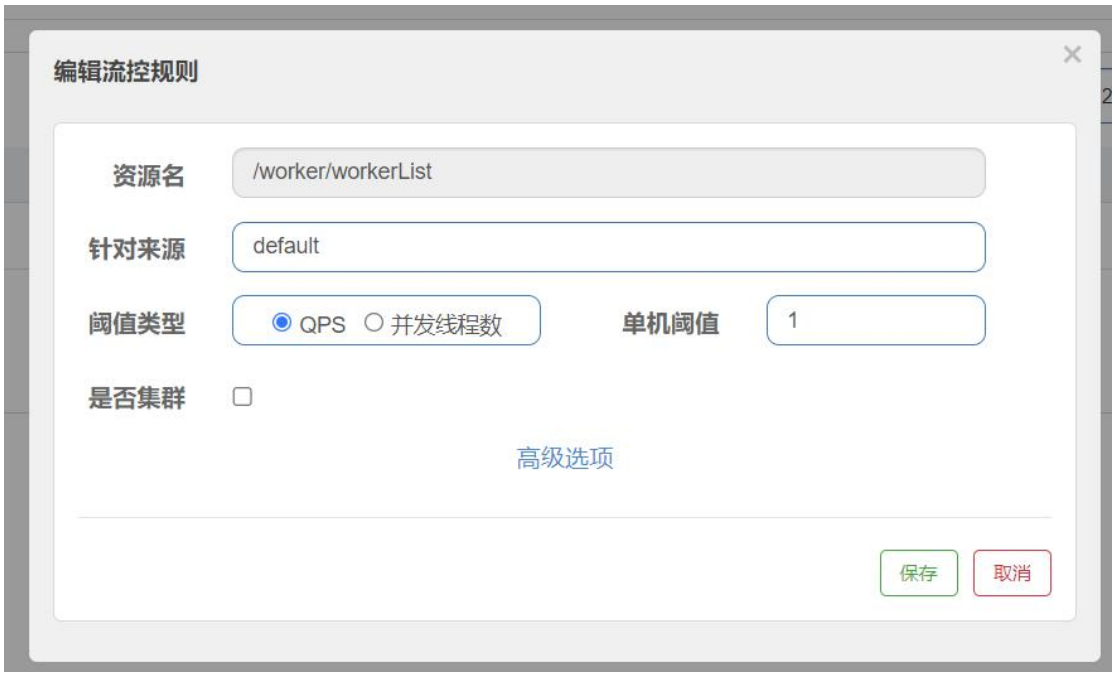
```
/**
 * FeignClient 中的 name   =   TestWorkerController 上的 RequestMapping
 *
 */
public interface IWorkerFeign {
    /**
     * 具体方法上的 RequestMapping
     */
    @RequestMapping("/workerList")
    public ResultVO getWorkerList();
}
```

3. 在 EngineerController 中添加一个接口

```
@RequestMapping("/get")
public ResultVO getEngineerList(){
    System.out.println("获取工程列表");
    return workerFeign.getWorkerList();
}
```

4. 启动 Engineer 服务、Worker 服务

5. 给/worker/workerList 添加一个流控策略如下:



6. 浏览器多次请求 Engineer 服务的 get 接口:
http://localhost:8088/engineer-serv/engineer/get，发现页面抛出 500 异常

7. 处理 500 异常，否则用户体验不行。学习 fallback 方法，可以理解为退路，当接口被限流了，我们要有一个应变计划。

现在 Engineer 服务中的 bootstrap.yml 中开启 OpenFeign 对 Sentinel 的支持，如下配置：

```
feign:
  sentinel:
    # 开启 Feign 对 Sentinel 的支持
    enabled: true
```

创建 fallback 包，在此包下创建一个实体类，名为 WorkerFallback

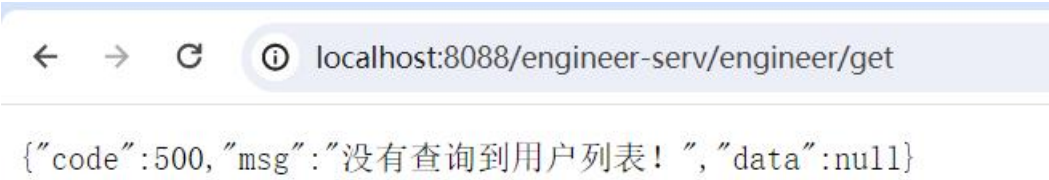
```
/***
 * @Author 高俊 QQ:1120934832
 * @Slogan 易途科技，精英启航
 */
@Service
public class WorkerFallback implements IWorkerFeign {
    @Override
    public ResultVO getWorkerList() {
        return ResultVO.failed("没有查询到用户列表！");
    }
}
```

在 IWorkerService 的@FeignClient 的注解上，添加一项配置，如下：

fallback = WorkerFallback.class

```
@FeignClient(name = "worker-service",path = "/worker",
        configuration = FeignLoggerConfig.class,fallback =
WorkerFallback.class)
```

再次请求，试试被限流后，OpenFeign 调用会返回什么



{"code":500,"msg":"没有查询到用户列表！","data":null}